

Homomorphic encryption is a form of encryption that allows computations to be performed on encrypted data without first having to decrypt it. The result of the computations are left in an encrypted form which, when decrypted, result in an output that is identical to that of the operations performed on the unencrypted data. Homomorphic encryption can be used for privacy-preserving outsourced storage and computation. This allows data to be encrypted and outsourced to commercial cloud environments for processing, all while encrypted.

As an example of a practical application of homomorphic encryption: encrypted photographs can be scanned for points of interest, without revealing the contents of a photo. However, observation of side-channels can see a photograph being sent to a point-of-interest lookup service, revealing the fact that photographs were taken.

Thus, homomorphic encryption eliminates the need for processing data in the clear, thereby preventing attacks that would enable an attacker to access that data while it is being processed, using privilege escalation.

For sensitive data, such as healthcare information, homomorphic encryption can be used to enable new services by removing privacy barriers inhibiting data sharing or increasing security to existing services. For example, predictive analytics in healthcare can be hard to apply via a third-party service provider due to medical data privacy concerns. But if the predictive-analytics service provider could operate on encrypted data instead, without having the decryption keys, these privacy concerns are diminished. Moreover, even if the service provider's system is compromised, the data would remain secure.

Characteristics

Homomorphic encryption is a form of encryption with an additional evaluation capability for computing over encrypted data without access to the secret key. The result of such a computation remains encrypted. Homomorphic encryption can be viewed as an extension of public-key cryptography, because ciphertexts can be manipulated algebraically to produce an

encrypted result corresponding to operations on the underlying plaintexts. Homomorphic refers to homomorphism in algebra: the encryption and decryption functions can be thought of as homomorphisms between plaintext and ciphertext spaces.

Homomorphic encryption includes multiple types of encryption schemes that can perform different classes of computations over encrypted data. The computations are represented as either Boolean or arithmetic circuits. Some common types of homomorphic encryption are partially homomorphic, somewhat homomorphic, leveled fully homomorphic, and fully homomorphic encryption:

Partially homomorphic encryption encompasses schemes that support the evaluation of circuits consisting of only one type of gate, e.g., addition or multiplication.

Somewhat homomorphic encryption schemes can evaluate two types of gates, but only for a subset of circuits.

Leveled fully homomorphic encryption supports the evaluation of arbitrary circuits composed of multiple types of gates of bounded (pre-determined) depth.

Fully homomorphic encryption (FHE) allows the evaluation of arbitrary circuits composed of multiple types of gates of unbounded depth and is the strongest notion of homomorphic encryption.

For the majority of homomorphic encryption schemes, the multiplicative depth of circuits is the main practical limitation in performing computations over encrypted data. Homomorphic encryption schemes are inherently malleable. In terms of malleability, homomorphic encryption schemes have weaker security properties than non-homomorphic schemes.

A cryptosystem that supports arbitrary computation on ciphertexts is known as fully homomorphic encryption (FHE). Such a scheme enables the construction of programs for any desirable functionality, which can be run on encrypted inputs to produce an encryption of the result. Since such a program need never decrypt its inputs, it can be run by an untrusted party

without revealing its inputs and internal state. Fully homomorphic cryptosystems have great practical implications in the outsourcing of private computations, for instance, in the context of cloud computing.

History

Homomorphic encryption schemes have been developed using different approaches. Specifically, fully homomorphic encryption schemes are often grouped into generations corresponding to the underlying approach.

Predecessors

The problem of constructing a fully homomorphic encryption scheme was first proposed in 1978, within a year of publishing of the RSA scheme. For more than 30 years, it was unclear whether a solution existed. During that period, partial results included the following schemes:

RSA cryptosystem (unbounded number of modular multiplications)

ElGamal cryptosystem (unbounded number of modular multiplications)

Goldwasser-Micali cryptosystem (unbounded number of exclusive or operations)

Benaloh cryptosystem (unbounded number of modular additions)

Paillier cryptosystem (unbounded number of modular additions)

Sander-Young-Yung system (after more than 20 years solved the problem for logarithmic depth circuits)

Boneh-Goh-Nissim cryptosystem (unlimited number of addition operations but at most one multiplication)

Ishai-Paskin cryptosystem (polynomial-size branching programs)

First generation

Craig Gentry, using lattice-based cryptography, described the first plausible construction for a fully homomorphic encryption scheme in 2009. Gentry's scheme supports both addition and multiplication operations on ciphertexts, from which it is possible to construct circuits for performing arbitrary computation. The construction starts from a somewhat homomorphic encryption scheme, which is limited to evaluating low-degree polynomials over encrypted data; it is limited because each ciphertext is noisy in some sense, and this noise grows as one adds and multiplies ciphertexts, until ultimately the noise makes the resulting ciphertext indecipherable.

Gentry then shows how to slightly modify this scheme to make it bootstrappable, i.e., capable of evaluating its own decryption circuit and then at least one more operation. Finally, he shows that any bootstrappable somewhat homomorphic encryption scheme can be converted into a fully homomorphic encryption through a recursive self-embedding. For Gentry's "noisy" scheme, the bootstrapping procedure effectively "refreshes" the ciphertext by applying to it the decryption procedure homomorphically, thereby obtaining a new ciphertext that encrypts the same value as before but has lower noise. By "refreshing" the ciphertext periodically whenever the noise grows too large, it is possible to compute an arbitrary number of additions and multiplications without increasing the noise too much.

Gentry based the security of his scheme on the assumed hardness of two problems: certain worst-case problems over ideal lattices, and the sparse (or low-weight) subset sum problem. Gentry's Ph.D. thesis provides additional details. The Gentry-Halevi implementation of Gentry's original cryptosystem reported a timing of about 30 minutes per basic bit operation. Extensive design and implementation work in subsequent years have improved upon these early implementations by many orders of magnitude runtime performance.

In 2010, Marten van Dijk, Craig Gentry, Shai Halevi and Vinod Vaikuntanathan presented a second fully homomorphic encryption scheme, which uses many of the tools of Gentry's construction, but which does not require ideal lattices. Instead, they show that the somewhat

homomorphic component of Gentry's ideal lattice-based scheme can be replaced with a very simple somewhat homomorphic scheme that uses integers. The scheme is therefore conceptually simpler than Gentry's ideal lattice scheme, but has similar properties with regards to homomorphic operations and efficiency. The somewhat homomorphic component in the work of Van Dijk et al. is similar to an encryption scheme proposed by Levieil and Naccache in 2008, and also to one that was proposed by Bram Cohen in 1998.

Cohen's method is not even additively homomorphic, however. The Levieil–Naccache scheme supports only additions, but it can be modified to also support a small number of multiplications. Many refinements and optimizations of the scheme of Van Dijk et al. were proposed in a sequence of works by Jean-Sébastien Coron, Tancreède Lepoint, Avradip Mandal, David Naccache, and Mehdi Tibouchi. Some of these works included also implementations of the resulting schemes.

Second generation

The homomorphic cryptosystems of this generation are derived from techniques that were developed starting in 2011–2012 by Zvika Brakerski, Craig Gentry, Vinod Vaikuntanathan, and others. These innovations led to the development of much more efficient somewhat and fully homomorphic cryptosystems. These include:

The Brakerski-Gentry-Vaikuntanathan (BGV, 2011) scheme, building on techniques of Brakerski-Vaikuntanathan;

The NTRU-based scheme by Lopez-Alt, Tromer, and Vaikuntanathan (LTV, 2012);

The Brakerski/Fan-Vercauteren (BFV, 2012) scheme, building on Brakerski's scale-invariant cryptosystem;

The NTRU-based scheme by Bos, Lauter, Loftus, and Naehrig (BLLN, 2013), building on LTV and Brakerski's scale-invariant cryptosystem;

The security of most of these schemes is based on the hardness of the (Ring) Learning With Errors (RLWE) problem, except for the LTV and BLLN schemes that rely on an overstretched variant of the NTRU computational problem. This NTRU variant was subsequently shown vulnerable to subfield lattice attacks, which is why these two schemes are no longer used in practice.

All the second-generation cryptosystems still follow the basic blueprint of Gentry's original construction, namely they first construct a somewhat homomorphic cryptosystem and then convert it to a fully homomorphic cryptosystem using bootstrapping.

A distinguishing characteristic of the second-generation cryptosystems is that they all feature a much slower growth of the noise during the homomorphic computations. Additional optimizations by Craig Gentry, Shai Halevi, and Nigel Smart resulted in cryptosystems with nearly optimal asymptotic complexity: Performing

T

$\{ \displaystyle T \}$

operations on data encrypted with security parameter

k

$$k$$

has complexity of only

$$T$$

$$p$$

$$o$$

$$l$$

$$y$$

$$l$$

$$o$$

$$g$$

$$($$

$$k$$

$$)$$

$$T \cdot \mathrm{polylog}(k)$$

. These optimizations build on the Smart-Vercauteren techniques that enable packing of many

plaintext values in a single ciphertext and operating on all these plaintext values in a SIMD fashion. Many of the advances in these second-generation cryptosystems were also ported to the cryptosystem over the integers.

Another distinguishing feature of second-generation schemes is that they are efficient enough for many applications even without invoking bootstrapping, instead operating in the leveled FHE mode.

Third generation

In 2013, Craig Gentry, Amit Sahai, and Brent Waters (GSW) proposed a new technique for building FHE schemes that avoids an expensive "relinearization" step in homomorphic multiplication. Zvika Brakerski and Vinod Vaikuntanathan observed that for certain types of circuits, the GSW cryptosystem features an even slower growth rate of noise, and hence better efficiency and stronger security. Jacob Alperin-Sheriff and Chris Peikert then described a very efficient bootstrapping technique based on this observation.

These techniques were further improved to develop efficient ring variants of the GSW cryptosystem: FHEW (2014) and TFHE (2016). The FHEW scheme was the first to show that by refreshing the ciphertexts after every single operation, it is possible to reduce the bootstrapping time to a fraction of a second. FHEW introduced a new method to compute Boolean gates on encrypted data that greatly simplifies bootstrapping and implemented a variant of the bootstrapping procedure. The efficiency of FHEW was further improved by the TFHE scheme, which implements a ring variant of the bootstrapping procedure using a method similar to the one in FHEW.

Fourth generation

In 2016, Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song (CKKS) proposed an approximate homomorphic encryption scheme that supports a special kind of fixed-point

arithmetic that is commonly referred to as block floating point arithmetic. The CKKS scheme includes an efficient rescaling operation that scales down an encrypted message after a multiplication. For comparison, such rescaling requires bootstrapping in the BGV and BFV schemes. The rescaling operation makes CKKS scheme the most efficient method for evaluating polynomial approximations, and is the preferred approach for implementing privacy-preserving machine learning applications. The scheme introduces several approximation errors, both nondeterministic and deterministic, that require special handling in practice.

A 2020 article by Baiyu Li and Daniele Micciancio discusses passive attacks against CKKS, suggesting that the standard IND-CPA definition may not be sufficient in scenarios where decryption results are shared. The authors apply the attack to four modern homomorphic encryption libraries (HEAAN, SEAL, HELib and PALISADE) and report that it is possible to recover the secret key from decryption results in several parameter configurations. The authors also propose mitigation strategies for these attacks, and include a Responsible Disclosure in the paper suggesting that the homomorphic encryption libraries already implemented mitigations for the attacks before the article became publicly available. Further information on the mitigation strategies implemented in the homomorphic encryption libraries has also been published.

Partially homomorphic cryptosystems

In the following examples, the notation

E

(

x

)

$$\mathcal{E}(x)$$

is used to denote the encryption of the message

x

$$x$$

.

Unpadded RSA

If the RSA public key has modulus

n

$\{\displaystyle n\}$

and encryption exponent

e

$\{\displaystyle e\}$

, then the encryption of a message

m

$\{\displaystyle m\}$

is given by

E

(
m
)
=

m

e

mod

n

$$\mathcal{E}(m)=m^e\pmod n$$

. The homomorphic property is then

E

(

m

1

)

.

E

(

m

2

)

=

m

1

e

m

2

e

mod

n

=

(

m

1

m

2

)

e

mod

n

=

E

(

m

1

.

m

2

)

$$\begin{aligned} \mathcal{E}(m_1) \cdot \mathcal{E}(m_2) &= m_1^e m_2^e \pmod{n} \\ &= (m_1 m_2)^e \pmod{n} \\ &= \mathcal{E}(m_1 \cdot m_2) \end{aligned}$$

ElGamal

In the ElGamal cryptosystem, in a cyclic group

G

$$G$$

of order

q

$$q$$

with generator

$$g$$

$$g$$

, if the public key is

$$($$

$$G$$

$$,$$

$$q$$

$$,$$

$$g$$

$$,$$

$$h$$

$$)$$

$$(G,q,g,h)$$

, where

$$h$$

$$=$$

$$g$$

$$x$$

$$h=g^x$$

, and

$$x$$

$$x$$

is the secret key, then the encryption of a message

$$m$$

$$m$$

is

$$E$$

$$($$

$$m$$

$$)$$

$$=$$

$$($$

g

r

,

m

.

h

r

)

$$\{\mathrm{mathcal{E}}\}(m)=(g^{\{r\}},m\cdot h^{\{r\}})$$

, for some random

r

∈

{

0

,

...

,

q

-

1

}

$$\{ \displaystyle r \in \{ 0, \ldots, q-1 \} \}$$

. The homomorphic property is then

E

(

m

1

)

.

E

(

m

2

)

=

(

g

r

1

,

m

1

.

h

r

1

)

(

g

r

2

,

m

2

.

h

r

2

)

=

(

g

r

1

+

r

2

,

(

m

1

.

m

2

)

h

r

1

+

r

2

)

=

E

(

m

1

.

m

2

)

.

$$\begin{aligned} E(m_1) \cdot E(m_2) &= (g^{r_1}, m_1 \cdot h^{r_1}) (g^{r_2}, m_2 \cdot h^{r_2}) \\ &= (g^{r_1+r_2}, (m_1 \cdot m_2) h^{r_1+r_2}) \end{aligned}$$

Goldwasser-Micali

In the Goldwasser-Micali cryptosystem, if the public key is the modulus

n

$$n$$

and quadratic non-residue

x

$$x$$

, then the encryption of a bit

$$b$$

$$b$$

is

$$E$$

$$($$

$$b$$

$$)$$

$$=$$

$$x$$

b

r

2

mod

n

$$\mathcal{E}(b)=x^br^2\pmod{n}$$

, for some random

r

∈

{

0

,

...

,

n

-

1

}

$$\{\displaystyle r\in \{0,\ldots ,n-1\}$$

. The homomorphic property is then

E

(

b

1

)

.

E

(

b

2

)

=

x

b

1

r

1

2

x

b

2

r

2

2

mod

n

=

x

b

1

+

b

2

(

r

1

r

2

)

2

mod

n

=

E

(

b

1

$\oplus$

b

2

)

.

$$\begin{aligned} \mathcal{E}(b_1) \cdot \mathcal{E}(b_2) &= x^{b_1} r_1^2 x^{b_2} r_2^2 \pmod{\phi(n)} \\ &= x^{b_1 + b_2} (r_1 r_2)^2 \pmod{\phi(n)} = \mathcal{E}(b_1 \oplus b_2). \end{aligned}$$

where

$$\oplus$$

$$\oplus$$

denotes addition modulo 2, (i.e., exclusive-or).

Benaloh

In the Benaloh cryptosystem, if the public key is the modulus

n

$\{\displaystyle n\}$

and the base

g

$\{\displaystyle g\}$

with a blocksize of

c

$\{\displaystyle c\}$

, then the encryption of a message

m

$$\{ \displaystyle m \}$$

is

E

(

m

)

=

g

m

r

c

$\bmod$

n

$$\mathcal{E}(m)=g^mr^c\pmod n$$

, for some random

r

$\in$

{
0
,
...
,
n
-
1
}

$$\{\displaystyle r\in \{0,\ldots ,n-1\}\}$$

. The homomorphic property is then

(

m

1

)

.

E

(

m

2

)

=

(

g

m

1

r

1

c

)

(

g

m

2

r

2

c

)

mod

n

=

g

m

1

+

m

2

(

r

1

r

2

)

c

mod

n

=

E

(

m

1

+

m

2

mod

c

)

.

$$\begin{aligned} E(m_1) \cdot E(m_2) &= (g^{m_1} r_1^c)(g^{m_2} r_2^c) \pmod{n} \\ &= g^{m_1+m_2} (r_1 r_2)^c \pmod{n} \\ &= E(m_1+m_2) \pmod{c} \end{aligned}$$

Paillier

In the Paillier cryptosystem, if the public key is the modulus

n

$\{\displaystyle n\}$

and the base

g

$\{\displaystyle g\}$

, then the encryption of a message

m

$\{\displaystyle m\}$

is

E

(

m

)

=

g

m

r

n

mod

n

2

$$\{\mathrm{mathcal{E}}\}(m)=g^{\{m\}}r^{\{n\}};\{\bmod{\{ \,;\}}\}n^{\{2\}}$$

, for some random

r

∈

{

0

,

...

,

$$\{0,1,\dots,n-1\}$$

$$\{r \in \{0,1,\dots,n-1\}\}$$

. The homomorphic property is then

$$E$$

$$($$

$$m$$

1

)

.

E

(

m

2

)

=

(

g

m

1

r

1

n

)

(

g

m

2

r

2

n

)

mod

n

2

=

g

m

1

+

m

2

(

r

1

r

2

)

n

mod

n

2

=

E

(

m

1

+

m

2

)

.

$$\begin{aligned} E(m_1) \cdot E(m_2) &= (g^{m_1} r_1^n)(g^{m_2} r_2^n) \pmod{n^2} \\ &= g^{m_1+m_2} (r_1 r_2)^n \pmod{n^2} = E(m_1+m_2). \end{aligned}$$

Other partially homomorphic cryptosystems

Okamoto–Uchiyama cryptosystem

Naccache–Stern cryptosystem

Damgård–Jurik cryptosystem

Sander–Young–Yung encryption scheme

Boneh–Goh–Nissim cryptosystem

Ishai–Paskin cryptosystem

Joye–Libert cryptosystem

Castagnos–Laguillaumie cryptosystem

Implementations

There are several open-source implementations of partially, somewhat and fully homomorphic encryption schemes. Second-generation and fourth-generation FHE scheme implementations typically operate in the leveled FHE mode (though bootstrapping is still available in some libraries) and support efficient SIMD-like packing of data; they are typically used to compute on encrypted integers or real/complex numbers. Third-generation FHE scheme implementations often bootstrap after each operation but have limited support for packing; they were initially used to compute Boolean circuits over encrypted bits, but have been extended to support integer arithmetics and univariate function evaluation. The choice of using a second-generation vs. third-generation vs fourth-generation scheme depends on the input data types and the desired computation.

Standardization

In 2017, researchers from IBM, Microsoft, Intel, the NIST, and others formed the open Homomorphic Encryption Standardization Consortium, which maintains a community security Homomorphic Encryption Standard.

See also

Client-side encryption – Type of encryption

Confidential computing – Privacy-enhancing computing technique

Format-preserving encryption – Method in cryptography

Homomorphic secret sharing

Homomorphic signatures for network coding

Private biometrics

Private set intersection – Secure multiparty computation cryptographic technique

Searchable symmetric encryption – System allowing searching of encrypted documents

Secure multi-party computation – Subfield of cryptography

Verifiable computing using a fully homomorphic scheme

References

External links

FHE.org Community (conference, meetup and discussion group)

Daniele Micciancio's FHE references

Vinod Vaikuntanathan's FHE references

"Alice and Bob in Cipherspace". American Scientist. September 2012. Retrieved 2018-05-08.

A list of homomorphic encryption implementations maintained on GitHub